Agnost AI: SDK & Ingestion Bug Report

Prepared by Kaushik (software engineer) · github.com/0xkaushik-ai · October 13, 2026

Summary

I integrated Agnost into a test agent from scratch, the way a new customer would, and sent ~150 scripted events to a free workspace. I then read the published SDK source and checked each suspicion with a small, targeted test. Where possible I wrote a fix and verified it against a local mock API, so nothing beyond those test events was sent to your servers.

16 issues: 2 High, 6 Medium, 8 Low. The two High issues lose or reorder customer data in normal use, including when following the docs' own advice to call `flush()`. A patch for the Python SDK fixes #1–4, #9 and #12; with it, a flaky mock network delivered **30/30** events in order, against **26/30 with 1–5 out of order** for 0.1.13.

#	Issue	Severity	Component	Status
1	flush() races the worker thread and reorders a conversation's events	**High**	Python SDK	Confirmed + fix verified
2	Any network error or 5xx drops the event (no retry)	**High**	Python & TS SDK	Confirmed + fix verified
3	track() blocks the caller on a synchronous HTTP request	**Medium**	Python SDK	Confirmed + fix verified
4	No client timestamp is sent; track(timestamp=...) is ignored	**Medium**	Python & TS SDK	Confirmed + fix verified
5	TS SDK takes over SIGINT/SIGTERM and exits with code 0	**Medium**	TS SDK (agnostai)	Confirmed
6	Ingestion returns 202 for unknown org IDs and empty bodies	**Medium**	Ingestion API	Confirmed
7	Ingestion accepts events for sessions that were never created	**Medium**	Ingestion API	Confirmed (API)
8	TS SDK sends up to 20 events concurrently; order not guaranteed	**Medium**	TS SDK (agnostai)	To verify (code)
9	debug=True produces no debug output	**Low**	Python SDK	Confirmed + fix verified
10	track() without conversation_id starts a new conversation every call	**Low**	Python SDK / docs	To verify (code)
11	identify() only affects conversations created afterwards	**Low**	Python SDK	To verify (code)
12	Properties sent as str(value): dicts become Python repr, not JSON	**Low**	Python SDK	Fix written
13	Python and TS SDKs send different wire formats for input/output	**Low**	Python & TS SDK	Confirmed (code)
14	Agent name is shown as a 'tool' on the Errors page	**Low**	Dashboard	Confirmed
15	Calling end() twice raises RuntimeError in the host app	**Low**	Python & TS SDK	Confirmed (code)
16	Package repo links return 404; docstrings import a non-existent module	**Low**	Packaging / docs	Confirmed

Test setup

Item	Detail
Packages	PyPI `agnost` 0.1.13; npm `agnostai` 0.2.0; npm `agnost` 0.2.1 (read only)
Environment	Python 3.13, Node 26, Linux; endpoint https://api.agnost.ai
Live data	35 scripted support-bot conversations (90 events) plus small targeted tests (users k1–k8). All users are synthetic, with no personal data.
Mock API	Local server with 20–80 ms random latency and a 503 on every 7th request; 3 conversations × 10 turns; 5 runs per SDK

Fix verification (mock API, 5 runs each)

Metric	Original 0.1.13	Patched
Events delivered (1 in 7 requests fail)	26 of 30	30 of 30
Events out of order	1–5 per run	0
Slowest track() call	39–81 ms	0–9 ms
Client timestamp sent	No	Yes

High severity

#1 flush() races the worker thread and reorders a conversation's events

High · Python SDK · Confirmed + fix verified

What happens. `EventProcessor.flush()` pulls events off the queue and sends them from the caller's thread while the background worker keeps sending from the same queue. Two senders run at once, so events from one conversation reach the API out of order. The docs tell users to call `flush()` or `shutdown()` before short-lived scripts exit, so following the docs triggers it.

Evidence. Live: a 3-turn conversation (test-user-031) showed its last user message first. Control: the same pattern with one sender at a time (users k1, k2) displayed correctly, including two turns in the same second, so the server orders correctly when events arrive in order. Mock API: 1–5 of 30 events out of order in every run with 0.1.13, and 0 with the patch.

Reproduce.

```
import agnost
agnost.init(ORG_ID)
for n in range(1, 11):
    agnost.track(user_id="u", conversation_id="c1",
                 input=f"turn {n}", output="ok")
agnost.flush()   # main thread and worker now send concurrently
```

Suggested fix. Only the worker thread sends. `flush()` waits on `queue.all_tasks_done` with a timeout instead of sending.

#2 Any network error or 5xx drops the event (no retry)

High · Python & TS SDK · Confirmed + fix verified

What happens. Both SDKs catch any send error, log it and drop the event. There is no retry on timeouts, connection resets or 5xx. In the Python SDK, if `capture-session` fails the conversation is still cached as created, so it is never retried and its events point to a session the server doesn't have.

Evidence. Mock API with one 503 in seven requests: 0.1.13 delivered 26 of 30 events in every run; the patch delivered 30 of 30.

Suggested fix. Retry network errors and 5xx with exponential backoff (the patch uses 3 attempts: 0.5 s, 1 s). Don't retry 4xx. Only cache a conversation once its creation succeeded.

Medium severity

#3 track() blocks the caller on a synchronous HTTP request

Medium · Python SDK · Confirmed + fix verified

What happens. The first `track()` of each conversation makes a synchronous `capture-session` POST in the caller's thread, with a 10 s timeout. In a web request handler this adds API latency to the user's response, and up to 10 s when the API is slow.

Evidence. Live: 312 ms for a new conversation against 1.7 ms for an existing one; 2,243 ms on the first call (cold connection). Mock: slowest call 39–81 ms with 0.1.13 and 0–9 ms with the patch.

Suggested fix. Queue session creation on the same FIFO worker as events. That also guarantees the session reaches the API before its events.

#4 No client timestamp is sent; track(timestamp=...) is ignored

Medium · Python & TS SDK · Confirmed + fix verified

What happens. `track()` computes a timestamp but never puts it in the payload, so the server stamps events on arrival. Late or concurrent events get the wrong time and order, and historical data can't be backfilled. The documented `timestamp` argument is silently ignored. The TS SDK sends none either.

Evidence. Payload built in `client.py` has no time field. Dashboard times match arrival, at one-second resolution.

Suggested fix. Send an ISO-8601 UTC `timestamp` (done in the patch) and have the server order by it, falling back to arrival time.

#5 TS SDK takes over SIGINT/SIGTERM and exits with code 0

Medium · TS SDK (agnostai) · Confirmed

What happens. `init()` installs SIGINT and SIGTERM handlers that call `process.exit(0)` once the SDK has flushed. The app's own shutdown logic (closing DB pools, finishing requests) is cut short, and the exit code reports success, which hides the signal from process managers and CI.

Evidence. With agnostai initialised, an app's 500 ms SIGTERM cleanup never finished and the exit code was 0. Without the SDK the cleanup finished and the exit code was 143.

Reproduce.

```
import * as agnost from "agnostai";
agnost.init(ORG_ID);
process.on("SIGTERM", () => setTimeout(() => process.exit(143), 500));
process.kill(process.pid, "SIGTERM");   // exits 0 at once
```

Suggested fix. Don't install signal handlers, or make it opt-in. Flush on `beforeExit` and document `await shutdown()`.

#6 Ingestion returns 202 for unknown org IDs and empty bodies

Medium · Ingestion API · Confirmed

What happens. `capture-session` and `capture-event` return **202** for an org ID that doesn't exist and for an empty JSON body. A customer with a mistyped ID gets no error anywhere, and their data is silently lost: a silent failure in the product built to catch silent failures.

Evidence. POST with org ID 00000000-0000-4000-8000-000000000000 and body `{}` returned 202.

Suggested fix. Validate the org ID (a cached lookup is cheap) and return 401/404. Return 400 for invalid bodies. Have the SDK log the error once.

#7 Ingestion accepts events for sessions that were never created

Medium · Ingestion API · Confirmed (API)

What happens. `capture-event` accepts an event whose session_id was never created. Combined with #2 and #8, events can end up with no conversation or user attached.

Evidence. POST of a valid event with a random session_id returned 202. How the dashboard shows it is still to verify.

Suggested fix. Create the session implicitly from the event, or reject the event with a clear error.

#8 TS SDK sends up to 20 events concurrently; order not guaranteed

Medium · TS SDK (agnostai) · To verify (code)

What happens. The TS EventProcessor sends up to 20 events at once (maxConcurrentSends = 20), and `begin()` starts `capture-session` without awaiting it. A turn's model, tool and agent events, and the session itself, can arrive in any order. With no client timestamp (#4), the server can't restore the order.

Evidence. Found in agnostai 0.2.0 dist/index.js. A live test (user k8-ts) was sent; the dashboard result is still to verify.

Suggested fix. Send events for the same session one after another (parallel across sessions only), await session creation, and send timestamps.

Low severity

#9 debug=True produces no debug output

Low · Python SDK · Confirmed + fix verified

What happens. `init(debug=True)` sets log levels but adds no handler, so debug and info messages never appear. Warnings and errors still print.

Evidence. debug=True plus 3 track() calls printed 0 characters.

Suggested fix. Attach a StreamHandler when debug is on and the app has no logging configured (in patch).

#10 track() without conversation_id starts a new conversation every call

Low · Python SDK / docs · To verify (code)

What happens. Without `conversation_id`, every `track()` creates a new random conversation. The docs' fire-and-forget example omits it, so multi-turn chats show up as many one-message conversations.

Evidence. session.py generates a new uuid per call. Live test (user k5-noconv) to verify.

Suggested fix. Document it clearly, or default to one conversation per user over an idle window.

#11 identify() only affects conversations created afterwards

Low · Python SDK · To verify (code)

What happens. `identify()` only stores traits locally. They're attached when a conversation is created, so identifying a user mid-conversation does nothing for that conversation.

Evidence. session.py. Live test (user k6-identify) to verify.

Suggested fix. Send traits to the API when identify() is called.

#12 Properties sent as str(value): dicts become Python repr, not JSON

Low · Python SDK · Fix written

What happens. Properties are sent as `str(value)`: `{'plan': 'pro'}` instead of JSON, and `True` instead of `true`. The TS SDK uses JSON, so the same data looks different depending on the SDK.

Evidence. client.py. Live test (user k4-props) to verify.

Suggested fix. json.dumps non-string values (in patch).

#13 Python and TS SDKs send different wire formats for input/output

Low · Python & TS SDK · Confirmed (code)

What happens. Python sends `args = "text"`; TS sends `args = {"input": "text"}`. Any server-side parsing has to handle both.

Evidence. Both SDK sources.

Suggested fix. Define one schema for both SDKs.

#14 Agent name is shown as a 'tool' on the Errors page

Low · Dashboard · Confirmed

What happens. The Errors page lists the agent (trippilot-test) under 'Tools impacted', because the SDK sends agent_name in primitive_name, the field MCP tool calls use.

Evidence. Dashboard screenshot.

Suggested fix. Send the agent name in its own field.

#15 Calling end() twice raises RuntimeError in the host app

Low · Python & TS SDK · Confirmed (code)

What happens. Calling `end()` twice, or setting properties after it, raises an exception in the host app. A double end() in an error path can crash a request.

Evidence. types.py (Python), Interaction class (TS).

Suggested fix. Log a warning and ignore the second call.

#16 Package repo links return 404; docstrings import a non-existent module

Low · Packaging / docs · Confirmed

What happens. PyPI links to github.com/agnostai/agnostai and npm agnostai links to github.com/agnost-ai/agnost-ai; both return 404. Docstrings show `import agnost_conversation`, which doesn't exist.

Evidence. GitHub API 404s; client.py and __init__.py.

Suggested fix. Fix the metadata links and docstrings.

Product observation (not a bug)

The 35 test conversations included 15 clear silent failures (users correcting the bot, repeating themselves or giving up) and 3 policy breaches. About an hour after ingestion, Intents and Violations were empty and AI Insights was still loading. This may be because a fresh workspace needs intents or SOPs configured first. If so, the dashboard could say so, because a new user who sends real traffic and sees nothing will assume the core feature doesn't work.

Repository: github.com/0xkaushik-ai/agnost-sdk-fixes

File	Contents
sdk/agnost/	agnost 0.1.13 with the fixes applied, one commit per fix
patch/	The same fixes as a single unified diff against 0.1.13
benchmark/compare.py	Mock-API harness that reproduces #1–3 and verifies the patch
live-tests/	Live checks used for #3, #5, #6, #7 and #9

Happy to walk through any of this, open these as PRs, or test further.